

Agricultural Analytics & Climate Forecasting Pipeline

1. Project Overview

This project looks at whether a business keeps enough stock to meet customer demand. The data has the demand and the available stock (availability) for 20 products on different dates. The goal is to find where supply falls short of demand and what that shortage is worth.

The scenario is a real working one: the report is first built on test environment data, and then has to be moved to the production environment data without breaking. The work was done in four stages: SQL Server for preparing the data, Power BI with DAX for the KPIs, MySQL for moving the data to a new database, and Power BI again to reconnect and publish the report.

2. Dataset Summary

- Inventory table columns: Order Date (DD/MM/YYYY), Product ID, Availability, Demand
- Products table columns: Product ID, Product Name, Unit Price (\$). It has 20 products, for example Phone Case, Smartwatch, Air Purifier, VR Headset and Laptop Stand.
- Joined table (new_table) columns: Order_Date_DD_MM_YYYY, product_id, availability, demand, product_name, unit_price
- Test data started as an Excel file. Production data was imported into SQL Server and then into MySQL.
- Joined table in MySQL (production data): 1,043 rows

3. Data Preparation using SQL Server

Test environment (database test_env)

1. Looked at the Products table and the inventory table.
2. Checked the distinct values of Order Date, Availability and Demand to understand the data.
3. Joined the two tables with a left join on Product ID, so each inventory row gets its product name and unit price.
4. Saved the result as new_table (select ... into).

Production environment (database prod)

1. Imported the production data and checked for null or blank order dates.
2. Listed the distinct Product IDs. IDs 21 and 22 were present, but the products table has only 20 products, so they had no matching product.

3. Remapped them as given in the project instructions, using UPDATE: 21 to 7 and 22 to 11.
4. Rebuilt new_table with the same left join, after the fix. If the join was done first, products 21 and 22 would have empty names and prices.

4. Moving the Data to MySQL

The same data was then moved to MySQL, so the report could be reconnected to a different database:

1. Installed MySQL Server, MySQL Workbench and the MySQL connector.
2. Imported the data into the MySQL database prod.
3. Rewrote the SQL in MySQL Workbench to create new_table (create table ... as select with a left join). This gave 1,043 rows with clean column names.
4. Fixed the errors on the way: wrong quotes, table names that need backticks, a misplaced semicolon, and safe update mode (error 1175) when running the UPDATE.

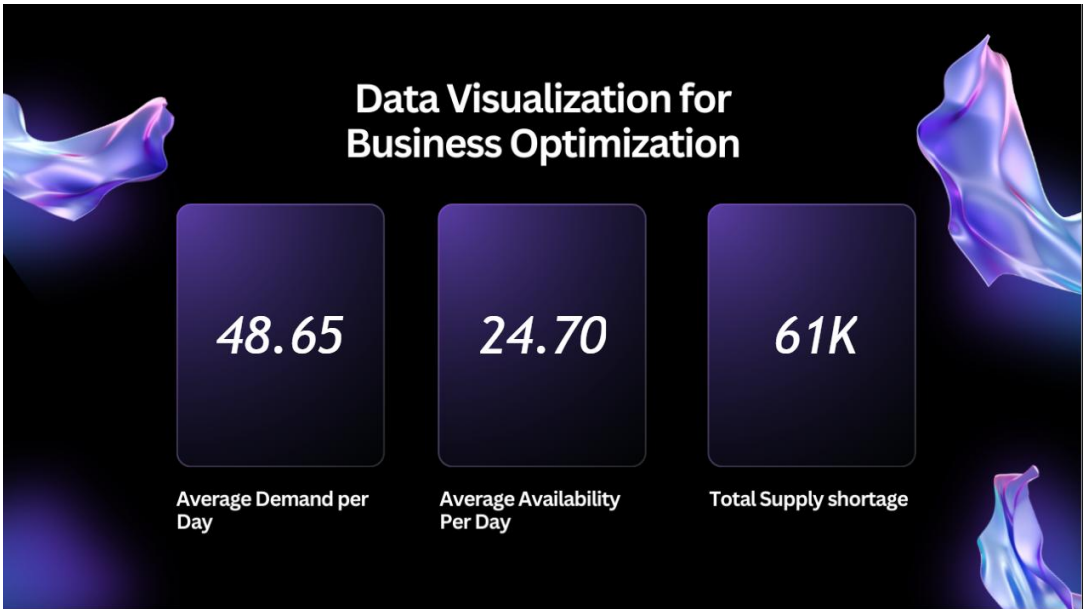
	SQL Server	MySQL
Names with spaces or symbols	[square brackets]	`backticks`
Create a table from a query	select ... into new_table	create table new_table as select ...
UPDATE without a key column	Allowed	Blocked by safe update mode until it is turned off

5. DAX Measures and Dashboard in Power BI

Measure	What it calculates
Total Availability	Sum of availability
Total Demand	Sum of demand
Total Supply Shortage	Total Demand minus Total Availability
Total Number of Days	Count of distinct order dates
Average Availability Per Day	Total Availability divided by Total Number of Days
Average Demand Per Day	Total Demand divided by Total Number of Days
Total Loss	Value of the shortages: rows with a negative Loss/Profit, multiplied by unit price, shown as a positive number
Total Profit	Value of the surplus stock: rows with a positive Loss/Profit, multiplied by unit price
Average Loss Per Day	Total Loss divided by Total Number of Days

A calculated column was added first: Loss/Profit = Availability - Demand. A negative value means demand was higher than stock (a shortage). A positive value means stock was higher than demand (a surplus). The measures Above use it.

Page 1: Demand and Availability

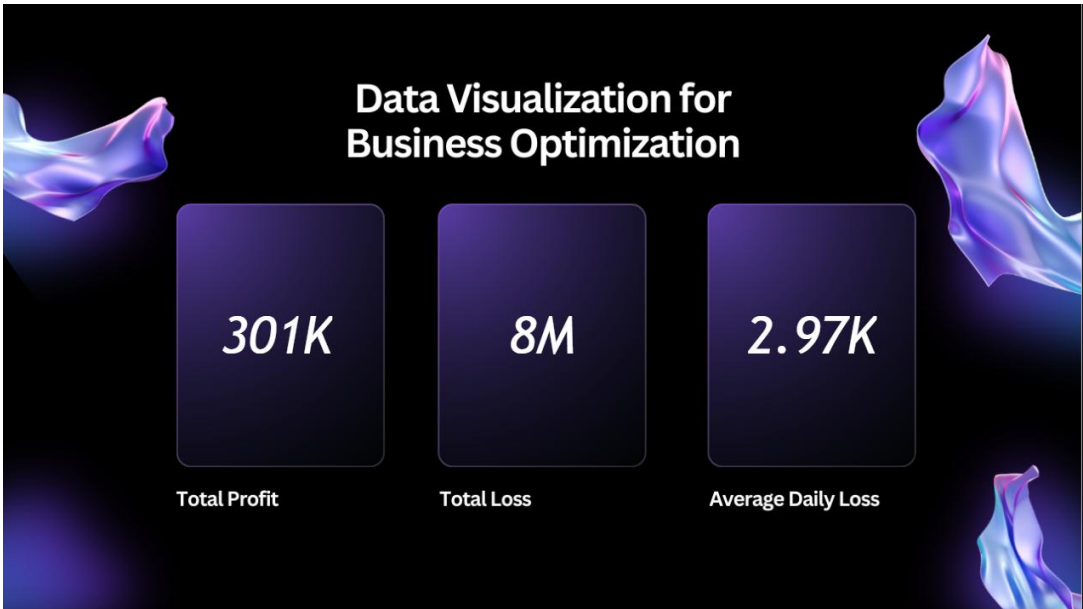


Dashboard: Page 1 (built on the test environment data)

KPI	Value
Average Demand per Day	48.65
Average Availability per Day	24.70
Total Supply Shortage	61K

- Average daily availability (24.70) is only about half of average daily demand (48.65).
- The total supply shortage is 61K units, so on average the business could not meet demand.

Page 2: Shortage and Surplus Value



Dashboard: Page 2 (built on the test environment data)

KPI	Value
Total Profit (surplus stock value)	301K
Total Loss (shortage value)	8M
Average Daily Loss	2.97K

- The shortage value (8M) is about 27 times the surplus value (301K), so stock shortage is the bigger problem, not overstock.
- On average, the shortage is worth about 2.97K per day.

Switching the Report to Production Data

After the report was built on test data, its source was switched to the production data in MySQL:

1. Pointed the report to the MySQL new_table using the Advanced Editor in Power Query.
2. Fixed the column names. MySQL returns lowercase names (product_id, availability, demand), but the "Changed Type" step still used the old names (Product_ID, Availability, Demand), which caused the error "The column Product_ID of the table wasn't found". Power Query is case sensitive, so the names have to match exactly.
3. Set the data types again: order date as date, IDs and quantities as whole numbers, product name as text, unit price as a number.
4. Validated the new data and published the report to the workspace.

Why the numbers change after the switch. The production data is on a different scale from the test file. In the production preview, demand is around 165 to 180 per record and availability is mostly between 1 and 15. In the test file, demand is around 40 to 55 and availability is up to about 45. So the KPI values on the production report will not match the test report, and the numbers on the two pages above belong to the test version.

6. Key Takeaways

Key takeaways

- Supply is behind demand: in the test data, daily availability was about half of daily demand, with a 61K unit shortage.
- Shortage is the main risk: the shortage value is about 27 times the surplus value, so the business loses more from stock-outs than it holds in extra stock.
- Data cleaning matters: product IDs that do not exist in the product list would leave blank names and prices, so they were fixed before the join.
- Reports can be moved between databases, but SQL syntax and column names need to be checked, or the report breaks.